LAPORAN PRAKTIKUM ALGORITMA DAN STRUKTUR DATA
ANALISIS KOMPLEKSITAS ALGORITMA : BIG O NOTATION, TIME & SPACE COMPLEXITY

Diampu Oleh:

M.Reza Zulman, S.ST. M.Sc

Disusun oleh :

ALFARABI ZIKRI 2025573010002

PROGRAM STUDI D4 TEKNIK INFORMATIKA

JURUSAN TEKNOLOGI INFORMASI DAN KOMPUTER

POLITEKNIK NEGERI LHOKSEUMAWE

2025/2026

DAFTAR ISI

DAFTAR ISI **2**

KATA PENGANTAR **3**

BAB I PENDAHULUAN **4**

1.1 Latar Belakang 4
1.2 Tujuan Praktikum 5
1.3 Alat dan Bahan 5

BAB II DASAR TEORI **6**

2.1 Konsep Time dan Space Complexity 6
2.2 Notasi Big O 6
2.3 Klasifikasi Kompleksitas Algoritma 7
2.4 Analisis Struktur Kode terhadap Kompleksitas 8

BAB III LANGKAH PRATIKUM **9**

3.1 Membuat Folder dan File JavaScript Melalui Git Bash 9
3.2 Menuliskan Program JavaScript 10

BAB IV PENUTUP **22**

4.1 Kesimpulan 22
4.2 Saran 23

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, laporan praktikum ini dapat diselesaikan dengan baik. Laporan ini disusun sebagai salah satu bentuk pemenuhan tugas pada mata kuliah Struktur Data dan Algoritma, khususnya pada materi Analisis Kompleksitas Algoritma.

Dalam laporan ini, penulis membahas mengenai konsep dasar Big O Notation, time complexity, dan space complexity, serta penerapannya dalam menganalisis dan mengoptimalkan algoritma menggunakan bahasa pemrograman JavaScript. Selain itu, laporan ini juga memuat hasil praktikum berupa implementasi dan perbandingan beberapa algoritma berdasarkan tingkat efisiensinya.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan, baik dari segi penulisan maupun isi. Oleh karena itu, penulis mengharapkan kritik dan saran yang membangun agar dapat menjadi bahan perbaikan di masa yang akan datang. Semoga laporan ini dapat bermanfaat bagi penulis maupun pembaca.

Buketrata, 29 April 2026

Alfarabi Zikri

BAB I
PENDAHULUAN

1.1 Latar Belakang

Dalam pengembangan perangkat lunak, efisiensi algoritma merupakan salah satu aspek penting yang harus diperhatikan. Seiring dengan meningkatnya ukuran data yang diproses, algoritma yang kurang efisien dapat menyebabkan waktu eksekusi yang lama serta penggunaan memori yang tidak optimal. Oleh karena itu, diperlukan suatu metode untuk menganalisis dan mengukur performa algoritma secara sistematis.

Salah satu metode yang digunakan adalah **analisis kompleksitas algoritma** dengan menggunakan **notasi Big O**. Notasi ini memungkinkan kita untuk mengetahui bagaimana pertumbuhan waktu eksekusi (*time complexity*) dan penggunaan memori (*space complexity*) suatu algoritma terhadap ukuran input. Dengan memahami konsep ini, seorang programmer dapat membandingkan berbagai pendekatan solusi dan memilih algoritma yang paling efisien.

Pada praktikum ini, dilakukan pengenalan dan penerapan konsep Big O melalui berbagai contoh algoritma seperti **$O(1)$, $O(n)$, $O(n^2)$, hingga $O(2^n)$**. Selain itu, praktikum juga membahas perbandingan performa algoritma secara empiris menggunakan pengukuran waktu eksekusi, serta analisis penggunaan memori dalam berbagai kasus. Praktikum ini juga melatih kemampuan dalam mengenali pola kompleksitas dan melakukan *refactoring* kode agar menjadi lebih optimal.

Dengan dilaksanakannya praktikum ini, diharapkan mahasiswa mampu memahami pentingnya efisiensi algoritma, serta dapat menerapkan analisis kompleksitas dalam pembuatan program yang lebih cepat dan efisien, terutama dalam menghadapi data berskala besar.

1.2 Tujuan Praktikum

Adapun tujuan dari Praktikum Setup Tools Pengembangan: Git, Node.js, dan Clone Repository adalah sebagai berikut :

1. Memahami konsep dasar **analisis kompleksitas algoritma** dalam pemrograman.
2. Menjelaskan perbedaan antara **time complexity** dan **space complexity**.
3. Memahami dan mampu menggunakan **notasi Big O** seperti O(1), O(log n), O(n), O(n log n), dan O(n^2).
4. Membandingkan efisiensi beberapa algoritma melalui **pengukuran waktu eksekusi secara empiris**.
5. Mengidentifikasi algoritma yang tidak efisien dan melakukan **refactoring** untuk meningkatkan performa.
6. Memahami hubungan antara penggunaan waktu dan memori (**time-space trade-off**) dalam algoritma.

1.3 Alat dan Bahan

Adapun alat dan bahan yang digunakan dalam praktikum ini adalah sebagai berikut:

1. Laptop atau komputer.
2. Koneksi internet.
3. Akun GitHub.
4. Git Bash Version 2.46.0.windows.1
5. Node.js v20.17.0.
6. Modul 3 Pratikum.

BAB II
DASAR TEORI

2.1 Konsep Time dan Space Complexity

Dalam analisis algoritma, terdapat dua aspek utama yang digunakan untuk mengukur efisiensi suatu program, yaitu *time complexity* dan *space complexity*. *Time complexity* digunakan untuk mengetahui seberapa banyak waktu yang dibutuhkan algoritma berdasarkan ukuran input (n), bukan dalam satuan detik, melainkan jumlah operasi yang dilakukan. Dengan demikian, algoritma dapat dibandingkan berdasarkan kecepatan pertumbuhannya ketika data semakin besar.

Sementara itu, *space complexity* mengukur jumlah memori tambahan yang digunakan selama proses eksekusi algoritma. Memori ini dapat berupa variabel, struktur data baru, maupun penggunaan *call stack* pada rekursi. Pemahaman terhadap kedua konsep ini penting karena dalam praktik sering terjadi keseimbangan antara waktu dan memori (*time-space trade-off*), sehingga programmer dapat menentukan algoritma yang paling efisien sesuai kebutuhan.

2.2 Notasi Big O

Notasi Big O merupakan metode yang digunakan untuk menggambarkan tingkat pertumbuhan waktu atau ruang yang dibutuhkan oleh suatu algoritma terhadap ukuran input (n). Big O tidak mengukur waktu secara langsung dalam satuan detik, melainkan menunjukkan bagaimana performa algoritma berubah ketika jumlah data semakin besar. Dengan menggunakan notasi ini, programmer dapat membandingkan efisiensi berbagai algoritma secara objektif tanpa bergantung pada spesifikasi perangkat keras.

Dalam penulisan Big O, terdapat beberapa aturan penyederhanaan, seperti mengabaikan konstanta dan mengambil suku dengan pertumbuhan paling

dominan. Misalnya, O(2n) disederhanakan menjadi O(n), dan $O(n^2 + n)$ menjadi $O(n^2)$. Hal ini dilakukan karena pada ukuran data yang besar, faktor dominan akan lebih berpengaruh terhadap performa algoritma dibandingkan nilai konstanta atau suku yang lebih kecil.

2.3 Klasifikasi Kompleksitas Algoritma

Kompleksitas algoritma dapat diklasifikasikan berdasarkan tingkat pertumbuhan waktu eksekusinya terhadap ukuran input (n). Beberapa kelas kompleksitas yang umum digunakan antara lain O(1) (konstan), O(log n) (logaritmik), O(n) (linear), O(n log n) (linearitmik), $O(n^2)$ (kuadratik), dan $O(2^n)$ (eksponensial). Setiap kelas memiliki karakteristik yang berbeda, di mana O(1) merupakan yang paling efisien karena tidak dipengaruhi oleh ukuran input, sedangkan $O(2^n)$ merupakan yang paling tidak efisien karena waktu eksekusinya meningkat sangat cepat seiring bertambahnya data.

Pemahaman terhadap klasifikasi ini sangat penting dalam memilih algoritma yang tepat. Algoritma dengan kompleksitas rendah seperti O(n) atau O(log n) lebih disarankan untuk digunakan pada data berskala besar, sedangkan algoritma dengan kompleksitas tinggi seperti $O(n^2)$ atau $O(2^n)$ sebaiknya dihindari kecuali untuk data yang kecil. Dengan mengetahui klasifikasi ini, programmer dapat memperkirakan performa algoritma dan menghindari penggunaan solusi yang tidak efisien.

2.4 Analisis Struktur Kode terhadap Kompleksitas

Analisis kompleksitas algoritma dapat dilakukan dengan mengamati struktur kode yang digunakan, seperti perulangan (*loop*) dan percabangan. Struktur kode yang berbeda akan menghasilkan tingkat kompleksitas yang berbeda pula. Misalnya, satu perulangan (*loop*) yang berjalan sebanyak n kali memiliki kompleksitas O(n), sedangkan perulangan bersarang (*nested loop*) dengan dua tingkat akan menghasilkan kompleksitas $O(n^2)$. Selain itu, perulangan yang membagi nilai input menjadi setengah setiap iterasi, seperti pada algoritma pencarian biner, memiliki kompleksitas O(log n).

Dengan memahami pola struktur kode, programmer dapat dengan mudah mengidentifikasi tingkat efisiensi suatu algoritma tanpa harus menjalankannya terlebih dahulu. Hal ini sangat membantu dalam proses pengembangan program, terutama untuk menghindari penggunaan algoritma yang tidak efisien. Oleh karena itu, kemampuan menganalisis struktur kode menjadi keterampilan penting dalam merancang algoritma yang optimal.

BAB III
LANGKAH PRATIKUM

3.1 Membuat Folder dan File JavaScript Melalui Git Bash

Langkah pertama dalam praktikum ini adalah membuat folder folder dan file yang akan dibuka di Visual Studio Code melalui Git Bash. Adapun langkah-langkah pembuatan akun GitHub adalah sebagai berikut:

1. Buat folder baru praktikum-5/ di dalam repository GitHub kamu dan inisialisasi npm project, lalu ikuti beberapa prompt dibawah gambar ini.

   mkdir praktikum-5
   cd praktikum-5

   Inisialisasi npm project.

2. Buat File di dalam folder pratikum-5

   touch 01-intro-kompleksitas.js

Ikuti Prompt diatas untuk membuat beberapa file sesuai dengan struktur berikut

```
praktikum-3/
── 01-intro-kompleksitas.js
├── 02-kelas-big-o.js
├── 03-space-complexity.js
├── 04-refactor.js
├── latihan-1.js
```

3. Buat folder dan file tugas.

mkdir tugas

touch tugas/tugas-1.js tugas/tugas-2.js

Ikuti Prompt diatas untuk membuat beberapa file sesuai dengan struktur berikut.

```
nama-repository/
└── praktikum-4/
    ├── package.json
    ├── 01-intro-kompleksitas.js
    ├── 02-kelas-big-o.js
    ├── 03-space-complexity.js
    ├── 04-refactor.js
    ├── latihan-1.js
    └── tugas/
        ├── tugas-1.js
        └── tugas-2.js
```

3.2 Menuliskan Program JavaScript

1. **01-intro-kompleksitas.js**

```javascript
// 01-intro-kompleksitas.js
// Mengukur perbedaan performa algoritma secara empiris

//fungsi bantu: ukur waktu eksekusi
function ukurWaktu(label, fn){
    const mulai =  Date.now();
    fn();
    const selesai = Date.now();
    console.log(`${label} : ${selesai - mulai} ms`);
}

const N = 100_100; // ukuran data: 100 ribu

// --- Algoritma A: o(n)/linear - loop satu kali
function jumlahkanLinear(n) {
    let total = 0;
    for (let i = 1; i <= N; i++) total += i;
```

```
    return total;
}

// --- Algoritma B: O(1) — rumus matematika ---
function jumlahkanRumus(n){
    return (n * (n + 1)) / 2;
}

// --- Algoritma C: O(n2) — loop bersarang ---
function cariPasangan(arr) {
    const pasangan = [];
    for(let i = 0; i < arr.length; i++) {
        for (let j = i + 1; j < arr.length; j++) {
            if (arr[i] + arr[j] === 10) pasangan.push([arr[i],
arr[j]]);
        }
    }
    return pasangan;
}

const data = Array.from({length: 5000}, (_, i) => i);

console.log('=== Perbandingan Waktu Eksekusi ===');
ukurWaktu('O(1) jumlahkanRumus ', () => jumlahkanRumus(N));
ukurWaktu('O(n) jumlahkanLinear ', () => jumlahkanLinear(N));
ukurWaktu('O(n2) cariPasangan ', () => cariPasangan(data));
console.log('\nHasil sama?', jumlahkanLinear(100) ===
jumlahkanRumus(100));
```

2. **02-kelas-big-o.js**

```
// 02-kelas-big-o.js
// Contoh konkret setiap kelas kompleksitas Big O
// ── O(1) — Konstan ──────────────────────────────
function ambilPertama(arr) { return arr[0]; }           // 0(1)
function tambahkanItem(arr, item) { arr.push(item)}     // 0(1)
function isGenap(n) {return n % 2 === 0; }

// ── O(log n) — Logaritmik ───────────────────────
function binarySearch(arr, target){
    let kiri = 0, kanan = arr.length - 1;
    let langkah = 0;
    while (kiri <= kanan){
```

```
        langkah++;3
        const tengah = Math.floor((kiri + kanan) / 2);
        if (arr[tengah] === target) {
            console.log(` Ditemukan di indeks ${tengah} serelang
${langkah} langkah`);
            return tengah;
        }
        if (arr[tengah] < target) kiri = tengah + 1;
        else kanan = tengah - 1;
    }
    return -1;
}
const data = [1,3,5,7,9,11,13,15];

console.log(binarySearch(data, 11));
console.log(binarySearch(data, 4));

//// ── O(n) — Linear ──────────────────────────────────
function cariMax(arr) {
    let maks = arr[0];
    for (const x of arr) if (x > maks) maks = x;
    return maks;
}

// ── O(n2) — Kuadratik ──────────────────────────────
function bubbleSortNaif(arr) {
    const a = [...arr];
    for(let i = 0; i < a.length; i++)
        for (let j = 0; j < a.length - i -1; j++)
            if (a[j] > a[j+1]) [a[j], a[j+1]] = [a[j+1], a[j]];
    return a;
}

// ── O(2n) — Eksponensial ───────────────────────────
function fibRekursif(n) {
    if (n <= 1) return n;
    return fibRekursif(n-1) + fibRekursif(n-2);  //dua panggil rekursif
}

// ── Demonstrasi ────────────────────────────────────
console.log('=== O(1) — selalu cepat ===');
console.log(ambilPertama([10,20,30,40,50])); // 10
console.log(isGenap(42)); // true


console.log('\n=== O(log n) — Binary Search ===');
const sorted = Array.from({length:1_000_000},(_,i)=>i);
```

```
binarySearch(sorted, 731_452); // cari di 1 juta data

console.log('\n=== O(n) — Linear Search ===');
console.log('Max dari 1000 elemen:',
cariMax(Array.from({length:1000},()=>Math.random()*1000|0)));

console.log('\n=== O(n2) — Bubble Sort (kecil saja) ===');
console.log(bubbleSortNaif([64, 34, 25, 12, 22, 11, 90]));

// Demonstrasi betapa lambatnya O(2n)
console.log('\nWaktu fib(35) O(2n):');
let t = Date.now(); fibRekursif(35); console.log(Date.now()-t, 'ms');

console.log('Waktu fib(35) O(n) memoization:');
const memo = {};
function fibMemo(n) {
    if (n <= 1) return n;
    if (memo[n]) return memo[n];
    return memo[n] = fibMemo(n-1) + fibMemo(n-2);
}
t = Date.now(); fibMemo(35); console.log(Date.now()-t, 'ms');
```

3. **04-space-complexity.js**

```
// 04-space-complexity.js

// --- O(1) Space: hanya pakai variabel tambahan konstan ---
function jumlahArray(arr) {
    let total = 0;
    for (const x of arr) total += x;
    return total;
}

// --- O(n) Space: membuat struktur baru sebesar input ---
function duplikasiArray(arr){
    const baru = [];
    for (const x of arr) baru.push(x * 2);
    return baru;
}

// --- O(n) Space: rekursi (call stack) ---
function faktorialRekursif(n) {
    if ( n <=  1) return 1;
    return n * faktorialRekursif(n - 1);
}
```

```
// --- O(1) Space: faktorial iteratif ---
function faktorialIteratif(n) {
    let hasil = 1;
    for (let i = 2; i <= n; i++) hasil *= i;
    return hasil;
}
const arr = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10];
console.log('Jumlah array :', jumlahArray(arr)); // O(1) space
console.log('Duplikasi array:', duplikasiArray(arr));
console.log('Faktorial 10 rekursif :', faktorialRekursif(4));
console.log('Faktorial 10 iteratif :', faktorialIteratif(4));

// Visualisasi: hitung elemen unik (O(n) space)
function hitungUnik(arr) {
 const seen = new Set(); // Set tumbuh hingga n elemen
 for (const x of arr) seen.add(x);
 return seen.size;
}

const dataAcak = [1,2,3,2,1,4,5,3,6,4,7];
console.log('Elemen unik:', hitungUnik(dataAcak)); // 7
```

4. **04-refactor.js**

```
// 04-array-method.js
// ================================================
// METHOD ITERASI: forEach, map, filter, reduce
// ================================================
const nilaiMahasiswa = [75, 90, 55, 82, 68, 95, 48, 77];

// --- 1. forEach: menjalankan sesuatu untuk setiap elemen ---
console.log("=== forEach: Tampilkan Semua Nilai ===");
nilaiMahasiswa.forEach((nilai, indeks) => {
  console.log(` Mahasiswa-${indeks + 1}: ${nilai}`);
});

// --- 2. map: transformasi setiap elemen menjadi nilai baru ---
// Konversi nilai angka ke grade huruf
const gradeHuruf = nilaiMahasiswa.map((nilai) => {
  if (nilai >= 90) return "A";
  if (nilai >= 80) return "B";
  if (nilai >= 70) return "C";
  if (nilai >= 60) return "D";
  return "E";
});
```

```
console.log("\n=== map: Nilai ke Grade ===");
console.log("Nilai :", nilaiMahasiswa);
console.log("Grade :", gradeHuruf);

// --- 3. filter: saring elemen yang memenuhi kondisi ---
const nilaiLulus = nilaiMahasiswa.filter((nilai) => nilai >= 60);
const nilaiGagal = nilaiMahasiswa.filter((nilai) => nilai < 60);

console.log("\n=== filter: Lulus dan Tidak Lulus ===");
console.log("Semua nilai :", nilaiMahasiswa);
console.log("Nilai lulus :", nilaiLulus);
console.log("Nilai Gagal :", nilaiGagal);

// --- 4. reduce: mereduksi array menjadi satu nilai ---
const totalNilai = nilaiMahasiswa.reduce((akumulator, nilai) => {
  return akumulator + nilai;
}, 0); //0 nilai awal akumulator

const rataRata = totalNilai / nilaiMahasiswa.length;

console.log("\n=== reduce: Statistik Nilai ===");
console.log("Total nilai    :", totalNilai);
console.log("Rata-rata      :", rataRata.toFixed(2));

// --- 5. Menggabungkan beberapa method (method chaining) ---
const rataRataNilaiLulus = nilaiMahasiswa
  .filter((nilai) => nilai >= 60)
  .reduce((acc, val, idx, arr) => {
    return acc + val / arr.length;
  }, 0);

console.log("\n=== Method Chaining ===");
console.log("Rata-rata nilai lulus:", rataRataNilaiLulus.toFixed(2));
```

5. **04-refactor.js**

```
// 05-refactor.js — Refactoring kode berperforma buruk
// ═══════════════════════════════════════════════
// SKENARIO 1: Cek duplikat dalam array
// ═══════════════════════════════════════════════
// BURUK: O(n²) — nested loop

// --- 1. Faktorial: n! = n × (n-1)! ---
function adaDuplikatLambat(arr) {
  for (let i = 0; i < arr.length; i++)
```

```
    for (let j = i + 1; j < arr.length; j++) if (arr[i] === arr[j])
return true;
  return false;
}
// BAIK: O(n) — gunakan Set
function adaDuplikatCepat(arr) {
  const seen = new Set();
  for (const x of arr) {
    if (seen.has(x)) return true;
    seen.add(x);
  }
  return false;
}

// ═══════════════════════════════════════════════
// SKENARIO 2: Frekuensi kemunculan elemen
// ═══════════════════════════════════════════════
// BURUK: O(n²) — indexOf di dalam loop

function frekuensiLambat(arr) {
  const unik = [];
  const hitung = [];
  for (const x of arr) {
    const idx = unik.indexOf(x); // O(n) di dalam loop O(n)
    if (idx === -1) {
      unik.push(x);
      hitung.push(1);
    } else hitung[idx]++;
  }
  return Object.fromEntries(unik.map((u, i) => [u, hitung[i]]));
}

function frekuensiCepat(arr) {
  const counter = {};
  for (const x of arr) counter[x] = (counter[x] || 0) + 1;
  return counter;
}

// ── Benchmark ──────────────────────────────
const besar = Array.from({ length: 20000 }, () =>
  Math.floor(Math.random() * 1000),
);

let t = Date.now();
adaDuplikatLambat(besar);
console.log("Duplikat LAMBAT O(n²):", Date.now() - t, "ms");
```

```
t = Date.now();
adaDuplikatCepat(besar);
console.log("Duplikat CEPAT O(n) :", Date.now() - t, "ms");

const kata = ["a", "b", "a", "c", "b", "a", "d"];
console.log("\nFrekuensi:", frekuensiCepat(kata));
```

6. **Analisis dan Refactor 1-tugas.js**

```
function intersectionLambat(arr1, arr2){
    const hasil = [];

    for (let i = 0; i < arr1.length;  i++) {
        for (let j = 0; j < arr2.length; j++) {
            if (arr1[i] === arr2[j]) {
                if (!hasil.includes(arr1[i])) {
                    hasil.push(arr1[i]);
                }
            }
        }
    }

    return hasil;
}


function intersectionCepat(arr1, arr2) {
    const setArr2 = new Set(arr2);
    const hasil = [];

    for (const x of arr1) {
        if (setArr2.has(x)) {
            hasil.push(x);
        }
    }

    return hasil;
}

function anagramCepat(arr) {
    const map = {};

    for (const kata of arr) {
        const key = sortKata(kata);
```

```
        if (!map[key]) map[key] = [];
        map[key].push(kata);
    }

    return Object.values(map);
}


function sortKata(kata) {
    return kata.split('').sort().join('');
}


// O(n³) — triple loop
function cekLambat(arr) {
    for (let i = 0; i < arr.length; i++) {
        for (let j = 0; j < arr.length; j++) {
            for (let k = 0; k < arr.length; k++) {
                if (arr[i] + arr[j] === arr[k] * arr[k]) {
                    return true;
                }
            }
        }
    }
    return false;
}


// O(n log n)  sorting + 2 pointer
function cekCepat(arr) {
    const a = [...arr].sort((x, y) => x - y);

    for (let k = a.length - 1; k >= 0; k--) {
        let left = 0;
        let right = k - 1;

        while (left < right) {
            const sum = a[left] + a[right];
            const target = a[k] * a[k];

            if (sum === target) return true;
            else if (sum < target) left++;
            else right--;
        }
    }
```

```
    return false;
}


console.log("Intersection:");
console.log(intersectionCepat([1,2,3,4],[3,4,5,6]));

console.log("\nAnagram:");
console.log(anagramCepat(['eat','tea','tan','ate','nat','bat']));

console.log("\nCek a+b=c²:");
console.log(cekCepat([2,7,11,15]));


const dataBesar = Array.from({length: 3000}, () =>
Math.floor(Math.random() * 1000));

let t = Date.now();
intersectionLambat(dataBesar, dataBesar);
console.log("\nIntersection Lambat:", Date.now() - t, "ms");

t = Date.now();
intersectionCepat(dataBesar, dataBesar);
console.log("Intersection Cepat :", Date.now() - t, "ms");


t = Date.now();
cekLambat([2,7,11,15,20]);
console.log("\nC Lambat:", Date.now() - t, "ms");

t = Date.now();
cekCepat([2,7,11,15,20]);
console.log("C Cepat :", Date.now() - t, "ms");
```

7. Visualisasi dan Pertembuhan Big O **2-tugas.js**

```
// TUGAS 2 — Visualisasi Pertumbuhan Big O
// ===============================================


// ─────────────────────────────────────────────
// O(1) — konstan
// ─────────────────────────────────────────────
function fn_O1(n) {
    return n + 1;
}


// ─────────────────────────────────────────────
// O(n) — linear
// ─────────────────────────────────────────────
function fn_On(n) {
    let total = 0;
    for (let i = 0; i < n; i++) {
        total += i;
    }
    return total;
}


// ─────────────────────────────────────────────
// O(n log n)
function fn_OnLogn(n) {
    let total = 0;

    for (let i = 0; i < n; i++) {
        for (let j = 1; j < n; j *= 2) { // log n
            total += j;
        }
    }

    return total;
}


// ─────────────────────────────────────────────
// O(n²) — nested loop
function fn_On2(n) {
    let total = 0;
```

```
    for (let i = 0; i < n; i++) {
        for (let j = 0; j < n; j++) {
            total += i + j;
        }
    }

    return total;
}


// ──────────────────────────────────────────────
// BENCHMARK
function benchmarkSemua(ukuranData) {
    for (const n of ukuranData) {
        console.log(`\n=== Ukuran Data: ${n} ===`);

        let t;

        t = Date.now();
        fn_O1(n);
        console.log("O(1)     :", Date.now() - t, "ms");

        t = Date.now();
        fn_On(n);
        console.log("O(n)     :", Date.now() - t, "ms");

        t = Date.now();
        fn_OnLogn(n);
        console.log("O(n log n):", Date.now() - t, "ms");

        t = Date.now();
        fn_On2(n);
        console.log("O(n²)    :", Date.now() - t, "ms");
    }
}


benchmarkSemua([100, 500, 1000, 5000, 10000]);


// O(1) hampir selalu 0 ms karena hanya 1 operasi.
// O(n) meningkat seiring bertambahnya n.
// O(n log n) lebih lambat dari O(n), tapi jauh lebih cepat dari O(n²).
// O(n²) meningkat sangat cepat dan menjadi paling lambat saat n besar.
```

BAB IV
PENUTUP

4.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, dapat disimpulkan bahwa analisis kompleksitas algoritma sangat penting dalam menentukan efisiensi suatu program. Dengan menggunakan notasi Big O, kita dapat mengetahui bagaimana pertumbuhan waktu eksekusi dan penggunaan memori terhadap ukuran data input.

Hasil percobaan menunjukkan bahwa algoritma dengan kompleksitas rendah seperti O(1) dan O(n) memiliki performa yang lebih baik dibandingkan algoritma dengan kompleksitas tinggi seperti $O(n^2)$ atau $O(2^n)$, terutama pada data berukuran besar. Selain itu, penggunaan teknik optimasi seperti refactoring dan pemanfaatan struktur data yang tepat, seperti Set atau Map, dapat secara signifikan meningkatkan efisiensi algoritma.

Dengan demikian, pemahaman terhadap konsep time complexity, space complexity, dan Big O sangat diperlukan agar dapat merancang algoritma yang optimal, efisien, dan sesuai dengan kebutuhan dalam pengolahan data.

4.2 Saran

Dalam pelaksanaan praktikum ini, disarankan agar mahasiswa lebih sering berlatih dalam menganalisis kompleksitas algoritma pada berbagai kasus yang berbeda. Pemahaman konsep seperti Big O, time complexity, dan space complexity akan lebih mudah dikuasai jika diiringi dengan praktik langsung dan pengujian terhadap program yang dibuat.

Selain itu, mahasiswa juga disarankan untuk membiasakan diri menulis kode yang efisien dengan memperhatikan struktur algoritma yang digunakan. Penggunaan teknik optimasi seperti refactoring serta pemilihan struktur data yang tepat, seperti Set atau Map, perlu terus dilatih agar dapat meningkatkan performa program, terutama saat menangani data dalam jumlah besar.